Functions

Help(print)

```
# Python built-in help signature
print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)
```

Prints the values to a stream, or to `sys.stdout` by default.

Optional keyword arguments:

`file` A file-like object (stream); defaults to the current `sys.stdout`.

`sep` String inserted between values, default is a space.

`end` String appended after the last value, default is a newline.

`flush` Whether to forcibly flush the stream.

User-Defined Functions

```
def least_difference(a, b, c):
  diff1 = abs(a - b)
  diff2 = abs(b - c)
  diff3 = abs(a - c)
  return min(diff1, diff2, diff3)

print(
  least_difference(1, 10, 100),
  least_difference(1, 10, 10),
  least_difference(5, 6, 7),
  # Python allows trailing commas
  # in argument lists.
)
```

How Functions Work

Function Header: Introduced by the `def` keyword. It declares the function name and its arguments `a, b, c`.

Indented Code Block: The statements following the `:` are executed when the function is called.

The return Keyword: Exits the function immediately and passes the resulting value back to the caller.

Pro Tip: *Python allows trailing commas in argument lists, making it easier to format and maintain multi-line function calls.*

Omitting the return Keyword

```
# 1. Previous function without return
def least_difference(a, b, c):
    diff1 = abs(a - b)
    diff2 = abs(b - c)
    # return statement is omitted!

# 2. What is the output here?
magic = print()
print(magic)
```

The "Implicit" Return

Default Python Behavior

If a function does not execute a `return` statement, it automatically exits and returns the special value **None**.

What print() Returns

The built-in `print()` function is called for its side effect (displaying text) but has nothing to return. Therefore, it returns **None**.

The Final Output

Executing the code snippet will first print a blank line (from the call to print) and then output:

```
None
```

Default Arguments

```
def greet(who="World"):
  print("Hello,", who)

greet()
greet(who="India")
# Argument name can be omitted

greet("DAU")
```

How Default Values Work

Default Values: Defined in the header with `who="World"`. When called without parameters, Python falls back to this default.

Named Arguments: Callers can explicitly specify parameter names to make the execution clear, e.g., `who="India"`.

Positional Arguments: When the argument list is simple and unambiguous, parameter names can be completely omitted.

Pro Tip: *Default arguments are ideal for defining fallback behaviors, reducing boilerplate in common use cases.*

Functions Applied to Functions

```
def mult_by_five(x):
return 5 * x

def call(fn, arg):
"""Call fn on arg"""
return fn(arg)

def squared_call(fn, arg):
"""Call fn on the result of calling fn on
arg"""
return fn(fn(arg))

print(
call(mult_by_five, 1),
squared_call(mult_by_five, 1),
sep='\n', # starts a new line
)
```

Higher-Order Functions

Functions as Arguments: Python functions are first-class citizens. They can be passed as inputs to other functions, like passing `mult_by_five` into `call`.

Nesting Function Calls: The `squared_call` function demonstrates behavior composition by executing the target function twice in a nested chain.

Expected Output: Executing this script prints the outputs on separate lines due to the `sep='\n'` parameter:

```
5 (from call)
25 (from squared_call)
```

Functions Applied to Functions

```
def mod_5(x):
    """Return the remainder of x after
    dividing by 5"""
    return x % 5

print(
    'Which number is biggest?',
    max(100, 51, 14),
    'Which number is the biggest modulo 5?',
    max(100, 51, 14, key=mod_5),
    sep='\n',
)
```

Functions as Keys

Custom Evaluation:
The **key** argument lets you pass a function. Python applies this function to each element before comparing them.

Modulo Comparison:
Instead of comparing 100, 51, and 14 directly, `max()` compares their remainder modulo 5:
`100 % 5 = 0 | 51 % 5 = 1 | 14 % 5 = 4`

Output Results:
Due to `sep='\n'`, the two print results are displayed on separate lines:

`100` (largest absolute value)
`14` (largest modulo 5 value)